ELSEVIER

Pattern Recognition Letters
journal homepage: www.elsevier.com

Secretary-style split search for decision trees

Mathias Valla*

Chaire DIALog, Institut Louis Bachelier, 28 Place de la Bourse, 75002 Paris, France

ABSTRACT

Exhaustive CART split search evaluates every admissible feature–threshold candidate at each node. We study whether secretary-inspired exploration followed by feature-level stopping can reduce this work while preserving predictive performance. The evaluated rules combine random-threshold exploration, exact within-feature scans, a blockwise rank-inspired heuristic, and a two-pass prophet-style heuristic. We compare complete depth-limited single-tree estimators with exhaustive CART and extremely randomized-tree baselines on 113 regression and 122 classification entries from PMLB, using fixed fivefold partitions and 100 estimator seeds. A paired hierarchical analysis separates within-entry randomization from variation across benchmark entries. Under the task-specific endpoints and margins used here, incomplete search met the displayed constraints more often for retained classification entries than for regression entries. Reductions in recorded gain evaluations did not consistently yield proportional wall-clock differences. Exact-feature stopping provided a conservative cross-task compromise; extremely randomized trees showed larger observed time savings with greater predictive loss. These results motivate matched ensemble experiments but do not establish a universal speedup.

Keywords: decision trees; split selection; secretary problem; early stopping; extremely randomized trees

1. Introduction

Classification and regression trees recursively select local feature–threshold splits that maximize impurity reduction [1, 2]. Continuous predictors can provide many admissible boundaries, making exhaustive split search a material part of training [3]. Random forests restrict the features considered at a node, while extremely randomized trees (ERT) additionally sample cut points [4, 5].

Prior acceleration strategies act at different levels. They prune underperforming features, compress or approximate candidate representations, or reduce the data processed during boosted-tree induction [6, 7, 8]. Streaming and dynamic trees instead decide splits from bounded-memory, approximate, or updateable summaries [9, 10, 11, 12]. Our setting is distinct: all node data are already in memory, every accepted gain is evaluated on the full node sample, and the algorithm decides how many features and exact thresholds to inspect before stopping.

Classical secretary rules explore a random prefix and accept the first subsequent record [13, 14]; sample-based secretary and prophet results provide related threshold constructions [15, 16]. Their guarantees do not transfer directly here: split candidates are grouped by feature, exact thresholds within a feature are sorted rather than randomly ordered, exploration fallbacks are retained, and some procedures revisit the node. We therefore use the terms *secretary-inspired*, *rank-inspired*, and *prophet-style* as design descriptions, not theorem claims.

We formulate and implement feature-ordered CART split-search rules that combine sampled-threshold exploration with exact selection. We compare them with exhaustive CART and ERT in complete trees, measuring wall-clock fit time and gain-evaluation effort. Paired multi-dataset inference treats datasets as the experimental units and reports hierarchical confidence intervals and tolerance-based reliability. The analysis includes both favorable and unfavorable time–prediction trade-offs.

Every fitted estimator in this study is a complete recursively grown tree, not a root split or stump. Fivefold evaluation fits five such trees for each method and seed. We deliberately isolate the within-node splitter before bagging introduces bootstrap resampling, aggregation, tree count, feature subsampling, and parallelism. Globally optimized tree methods jointly alter topology and split selection and therefore address a different estimand [17, 18].

2. Methods

2.1. Split search and stopping rules

At a node $\mathcal{N}$ with n samples, let m be the pre-scan feature budget after excluding features already known to be constant from ancestor nodes, and let C_j be the number of distinct adjacent-value boundaries on feature j. Further constant features can be discovered during the node scan. For impurity I, a split (j, s) has gain

$$G_{j,s} = I(\mathcal{N}) - \frac{n_{\mathrm{L}}}{n} I(\mathcal{N}_L) - \frac{n_{\mathrm{R}}}{n} I(\mathcal{N}_R). \tag{1}$$

Regression uses mean-squared-error impurity; classification uses Gini or entropy. An exact feature scan sorts node samples by that feature and updates the criterion incrementally across admissible boundaries.

The early-stop trees use all available features in this benchmark. Features are drawn without replacement in random order. An exploration fraction f gives a target of

$$r = \max\{1, \min[m, \mathrm{round}(fm)]\}$$

non-constant features encountered in that order. A feature found constant at the current node is skipped without reducing this

*Corresponding author: mathias.valla@gmail.com

Input: node samples, randomized pre-scan feature budget $(j_1, \ldots, j_m)$, exploration fraction f.
1. Set $r \leftarrow \mathrm{clamp}(\mathrm{round}(fm), 1, m)$, $\tau \leftarrow -\infty$, and fallback $\leftarrow \varnothing$.
2. Traverse the randomized budget; skip features found constant without advancing t. For each encountered non-constant feature j_t:
   a. If $t \leq r$, compute the variant's exploration score on j_t; if it exceeds τ, update τ and the fallback.
   b. Otherwise, exact-scan j_t for the variant's qualifying score; return it as soon as the feature qualifies.
3. Return the exploration fallback if no later feature qualifies.

Algorithm 1: Common feature-ordered framework for S, S_{all}, and S^2.

target. The archived schedules are $f = 1/e$, $f = 0.1$, $f = 1/\log N$ using full-dataset size N, and $f = 1/\sqrt{n}$ recomputed at each node. These are feature-prefix fractions, not candidate counts proportional to n. For S^2, f controls only the outer feature prefix; its inner sampled-threshold fraction is fixed at $1/e$.

The sampled-threshold rule $S(f)$ draws approximately fC_j continuous thresholds over each exploration feature's observed range and sets τ to the best sampled gain. Each later feature is scanned exactly; the method accepts the first feature whose best qualifying split exceeds τ. Thus, stopping is at the feature level, not at the first qualifying threshold. The exact-feature rule $S_{\mathrm{all}}(f)$ instead uses the exact maximum of every visited feature in both phases. The calibration ablation $S^2(f)$ samples thresholds on each exploration feature and then performs a complete exact scan above the sampled level; it can therefore evaluate more gains than exhaustive search on an exploration feature.

The blockwise rank-inspired heuristic scans sorted exact boundaries in blocks of $\lfloor\sqrt{n}\rfloor$, represents each block by its maximum, and independently assigns it to exploration with probability $1/e$ or to selection otherwise. This is not the block-rank algorithm of Nuti and Vondrák [15]. The prophet-style heuristic first draws one ERT-style continuous threshold per feature, sets τ to their maximum gain, then replays the feature order and exact-scans until a feature has a qualifying split. ERT draws one continuous threshold per selected feature and returns the best sampled split [5]. Detailed, code-linked pseudocode for every rule is given in Supplementary Section S2.

2.2. Computational scope

At a fixed node, exhaustive CART evaluates at most $\sum_j C_j$ gains and sorts every visited feature, giving a comparison-scale cost $O(mn \log n + \sum_j C_j)$. ERT avoids sorting and evaluates at most m gains, with $O(mn)$ partition work. The gain counts of S, S_{all}, and the blockwise heuristic do not exceed exhaustive search at the same node, although their sorting and control-flow costs remain. The extra sampled evaluations in S^2 can exceed that reference; the prophet-style rule adds a full min/max and partition pass before exact replay. Full-tree effort need not preserve node-level ordering because different splits create different descendants.

The recorded effort metric counts admissible proxy-gain evaluations. It excludes sorting, candidate counting, min/max passes, random-threshold sorting, partitioning, calibration, memory traffic, and final split application. CART and ERT effort is reconstructed for successful internal-node searches, whereas early-stop counters also include failed split calls that produce leaves. The total-tree effort comparisons are therefore conservative diagnostics for early stopping; per-call normalization is not reported. Effort saved and wall-clock time saved remain complementary rather than interchangeable outcomes.

3. Experimental design

3.1. Benchmark and reproducibility

We analyzed dense, finite-valued PMLB entries [19] that completed the archived protocol. The retained corpus contains 113 regression and 122 classification entries; Gini and entropy use the same classification corpus. Table 1 reports quartiles and ranges, and the supplement provides a machine-readable inventory. The exact sharded invocation was not retained; because the executable default applies $np \leq 10^6$, the archive cannot verify that this cap was disabled.

Each method used the same deterministic fivefold partitions and common tree settings, including `max_depth=20`. For each method and seed, five fold-specific single trees were fitted and their outcomes averaged. Randomized estimators used seeds 42–141, giving 100 seeded repetitions on fixed folds; the folds are not independent replicates. ERT was evaluated with $m_{\mathrm{try}} \in \{1, p/3, 2p/3, p\}$. Reported fit time is estimator-only time from scikit-learn's cross-validation routine [20]. Methods were executed in a fixed CART-first order, so observed time contrasts include any systematic order effect. Regression performance is RMSE and classification performance is weighted F1.

The archive records Python 3.10.18, NumPy 1.26.4, scikit-learn 1.7.2, treeple 0.10.3, and macOS 15.2 on ARM64 with eight CPUs. The benchmark host was an Apple M3 MacBook Air with 16 GB memory. The revised analysis bundle hashes all 300 raw run files and the audited benchmark, splitter, reference-splitter, build, analysis, and test sources. The exact temporary compiled extension and sharded shell invocation were not retained, so the supplement states this provenance limitation rather than asserting bitwise executable identity.

3.2. Estimands and inference

PMLB entries are the top-level experimental units [21]. Within each entry, every method is paired with exhaustive CART by seed. An entry point is the median over its complete paired seed block. Time saved is $100[1 - 1/\,\mathrm{median}(t_{\mathrm{CART}}/t_m)]$. Regression loss is $100\,\mathrm{median}(1 - \mathrm{RMSE}_{\mathrm{CART}}/\mathrm{RMSE}_m)$; classification loss is $100\,\mathrm{median}(\mathrm{F1}_{\mathrm{CART}} - \mathrm{F1}_m)$ in F1 percentage points. One regression dataset, `564_fried`, lacks seed 45 (run 4); that seed is excluded for all methods without imputation.

We report paired two-stage percentile-bootstrap intervals with 10,000 replicates and analysis seed 20260810 [22]. Seed identifiers are jointly resampled across methods within an entry, then

Table 1. Retained PMLB datasets: Q1/median/Q3, with ranges in parentheses.

Task	N	Observations n	Features p	Classes
Regression	113	250/500/1,000 (47–40,768)	5/10/25 (2–124)	–
Classification	122	202/736/3,196 (32–58,000)	6.25/13/29 (2–240)	2/2/4 (2–26)

complete entries are resampled with equal weight. Two-sided 95% intervals cover entry medians, the equal-entry-weight mean of medians used as each centroid, and the cross-entry median. Revision-designated representatives are compared using Friedman tests and paired Wilcoxon tests versus exhaustive CART with Holm correction. A primary one-sided exact test asks whether more than half of entries simultaneously have positive median time saving and loss below the task-specific 1-unit margin; 0.5 and 2.5 form one multiplicity-adjusted sensitivity family. A family-balanced regression analysis groups related Friedman entries and exact aliases and reaches the same primary decision. Moving-block bootstraps assess timing dependence but cannot remove fixed method-order bias. PMLB is curated, so intervals quantify benchmark heterogeneity rather than universal population coverage.

4. Results

4.1. Time–prediction trade-offs

Figure 1 separates conservative low-loss methods from aggressively randomized baselines. Under the chosen task-specific endpoints and margins, incomplete search met constraints more often among retained classification entries than regression entries. The exact-feature rule $S_{\mathrm{all}}(1/e)$ provides a conservative secretary-inspired compromise across tasks, while ERT shows larger observed time savings at greater predictive loss. The sampled $S(1/e)$ and prophet-style rules often reduce gain evaluations substantially, but their additional sorting or replay is not accompanied by a proportional wall-clock difference.

Table 2 reports the same named estimands and their 95% hierarchical intervals. Classification losses are absolute weighted-F1 percentage points; regression losses are relative RMSE percentages, so their numerical magnitudes are not directly comparable. The archived parametric calibration screen is omitted from the main analysis because the revision audit found defects in both its regression and classification calibration; it is retained only as a supplementary implementation audit. This exclusion is conditional on the original all-method complete-case corpus, and its possible influence on corpus inclusion cannot be reconstructed.

For regression, the equal-entry-weight mean observed time-saving estimate for $S_{\mathrm{all}}(1/e)$ was 16.44% (95% interval 15.45–17.20), with 2.67% mean RMSE loss (1.97–3.46). In classification, the corresponding time-saving estimates were 12.74% (11.31–14.07) for Gini and 12.59% (11.20–13.85) for entropy, while mean F1 losses were 0.07 (-0.12–0.25) and 0.03 (-0.17–0.20) percentage points. ERT with $m_{\mathrm{try}} = p$ showed about 38% mean time saving in both classification tasks, with mean F1 losses 0.42 (-0.03–0.91) and 0.40 (0.02–0.75) percentage points. These time contrasts retain the fixed-order caveat.

4.2. Reliability and observed regimes

Figure 2 gives every entry equal weight rather than pooling seed rows. At required time savings of 0% or 10% on this benchmark, $S_{\mathrm{all}}(1/e)$, $S^2(1/e)$, and $S(1/e)$ are the leading secretary-style variants; their ordering depends on task and loss tolerance. At higher required savings, prophet-style and randomized methods can rank ahead. Supplementary Figures S3–S7 report within-entry variability and size-stratified reliability.

In the primary simultaneous-success tests, no regression alternative was supported after Holm correction. For both classification criteria, $S^2(1/e)$, $S_{\mathrm{all}}(1/e)$, and ERT with $m_{\mathrm{try}} = p$ were supported. At the stricter 0.5-unit sensitivity, $S_{\mathrm{all}}(1/e)$ remained supported for both criteria and $S^2(1/e)$ was additionally supported for Gini. Full counts and adjusted tests are reported in Supplementary Table S4.

Figure 3 colors each observed entry by the fastest method whose paired-seed median loss meets the displayed margin. Exhaustive CART is always feasible at zero loss; ERT with $m_{\mathrm{try}} = 1$ can be fastest yet inadmissible because of loss. Under these task-specific endpoints, classification contains broader regions where early stopping or ERT satisfies tight margins, whereas regression more often retains exhaustive or conservative exact-feature search. The interpolated background is descriptive, not a validated decision rule for unseen datasets.

The 1-unit row gives a concrete view of these regimes. ERT was the fastest admissible family for 62/122 Gini and 68/122 entropy entries. It was selected for 23/32 and 22/32 classification entries with $n \geq 3196$, and for 22/31 entries under each impurity when $p/n \leq 0.00769$. Regression retained exhaustive CART for 63/113 entries overall and 24/43 entries with $n \geq 1000$; S_{all} was selected for 13/113. The cutoffs are corpus quartiles and are descriptive, not deployment thresholds.

5. Discussion and conclusion

This study evaluates local split-search mechanisms with uncertainty estimates and reports both favorable and unfavorable results. Reduced gain counts alone are insufficient: sorting, candidate counting, partitioning, calibration, replay, and changed tree topology can dominate observed fit time. This explains differences between Figure 1 and the effort analyses in Supplementary Figures S13–S14 without equating the two metrics.

The implementation audit also narrows the theoretical interpretation. Randomization occurs over features, while exact thresholds are scanned in sorted value order; exploration fallbacks and revisits violate classical irrevocability. No classical secretary or prophet approximation ratio is claimed. S^2 is a calibration ablation rather than a cheaper nested stopping rule, and the blockwise method is rank-inspired rather than the published block-rank algorithm.

The evidence is conditional on dense PMLB datasets that completed the protocol, one hardware/software configuration, fixed folds, and complete depth-limited individual trees. Timing rankings can change with implementation and processor. At the 1-unit margin, ERT was most often admissible and fastest among larger, low-p/n classification entries, whereas regression more often retained exhaustive CART; these are descriptive quartile patterns. The benchmark does not establish ensemble behavior. A matched bagging study is a logical next step because aggregation may attenuate occasional suboptimal splits and repeated node-level savings may accumulate. Such a study must measure resampling, tree count, feature subsampling, aggregation, and parallel execution directly.

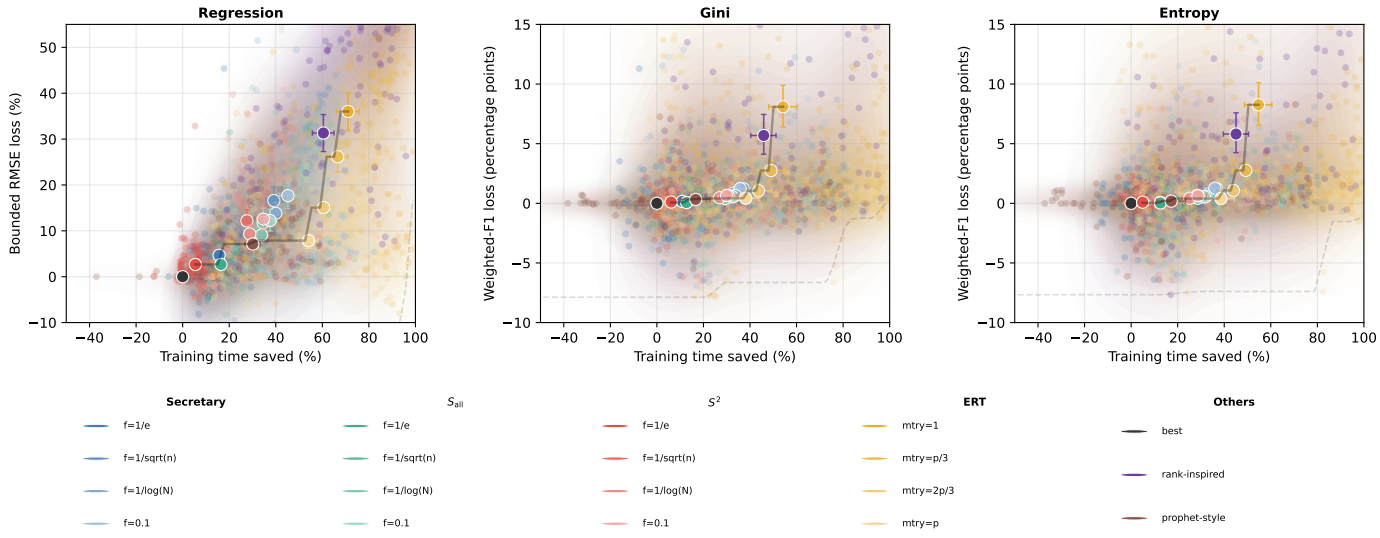

Fig. 1. Time saved and predictive loss relative to exhaustive CART. Small points are paired-seed entry medians; large points are equal-weight entry centroids with marginal 95% hierarchical bootstrap intervals.

Table 2. Representative-method summaries with hierarchical 95% bootstrap intervals. Centroids average entry-level run medians; classification losses are weighted-F1 percentage points and all other entries are percentages.

Task	Method	Centroid time saved	Centroid loss	Median loss	Median effort saved
Regression	Exhaustive	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]
Regression	S (f=1/e)	15.67 [14.10, 16.96]	4.66 [3.56, 5.84]	3.88 [2.88, 4.90]	46.85 [45.87, 47.47]
Regression	S^2 (f=1/e)	5.46 [4.45, 6.27]	2.67 [1.92, 3.49]	1.75 [1.33, 2.56]	1.88 [0.35, 3.56]
Regression	S_{all} (f=1/e)	16.44 [15.45, 17.20]	2.67 [1.97, 3.46]	1.81 [1.39, 2.38]	17.42 [16.47, 18.57]
Regression	Rank-inspired	60.46 [55.74, 65.03]	31.33 [27.29, 35.35]	31.91 [24.87, 39.28]	90.76 [88.18, 92.08]
Regression	Prophet-style	30.13 [26.74, 33.27]	7.12 [5.83, 8.17]	6.43 [4.28, 8.17]	58.38 [55.99, 60.49]
Regression	ERT ($m_{\text{try}} = 1$)	71.05 [66.02, 75.71]	36.02 [31.88, 40.02]	36.67 [30.65, 44.01]	98.93 [98.51, 99.08]
Regression	ERT ($m_{\text{try}} = p/3$)	66.59 [61.86, 71.00]	26.15 [22.85, 29.34]	28.37 [23.89, 33.25]	96.70 [96.47, 96.97]
Regression	ERT ($m_{\text{try}} = 2p/3$)	60.50 [55.99, 64.78]	15.08 [12.85, 17.36]	15.23 [12.05, 17.85]	92.95 [92.33, 93.53]
Regression	ERT ($m_{\text{try}} = p$)	54.04 [49.62, 58.16]	7.86 [6.20, 9.25]	8.46 [6.59, 10.23]	88.80 [87.67, 89.54]
Classification (Gini)	Exhaustive	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]
Classification (Gini)	S (f=1/e)	10.89 [8.38, 13.54]	0.16 [-0.15, 0.50]	0.10 [0.00, 0.24]	42.45 [38.45, 45.32]
Classification (Gini)	S^2 (f=1/e)	6.13 [4.97, 7.08]	0.11 [-0.07, 0.30]	0.05 [0.00, 0.16]	-1.10 [-5.02, 0.74]
Classification (Gini)	S_{all} (f=1/e)	12.74 [11.31, 14.07]	0.07 [-0.12, 0.25]	0.03 [0.00, 0.11]	15.27 [14.14, 16.27]
Classification (Gini)	Rank-inspired	45.89 [40.31, 51.20]	5.69 [4.11, 7.45]	2.67 [1.57, 3.61]	71.89 [65.37, 75.59]
Classification (Gini)	Prophet-style	16.64 [10.11, 22.80]	0.33 [0.05, 0.62]	0.03 [0.00, 0.26]	41.50 [31.74, 51.89]
Classification (Gini)	ERT ($m_{\text{try}} = 1$)	54.19 [48.08, 60.21]	8.10 [6.39, 9.90]	4.57 [3.22, 7.11]	96.34 [92.80, 97.69]
Classification (Gini)	ERT ($m_{\text{try}} = p/3$)	49.04 [43.17, 54.75]	2.75 [1.99, 3.55]	1.74 [1.28, 2.60]	89.25 [83.75, 93.86]
Classification (Gini)	ERT ($m_{\text{try}} = 2p/3$)	43.69 [38.04, 49.27]	1.05 [0.50, 1.63]	1.01 [0.40, 1.33]	84.10 [76.03, 90.52]
Classification (Gini)	ERT ($m_{\text{try}} = p$)	38.14 [32.37, 43.74]	0.42 [-0.03, 0.91]	0.29 [0.00, 0.61]	84.57 [77.92, 88.70]
Classification (Entropy)	Exhaustive	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]
Classification (Entropy)	S (f=1/e)	12.59 [9.80, 15.29]	0.08 [-0.20, 0.33]	0.10 [0.00, 0.34]	41.95 [38.18, 44.00]
Classification (Entropy)	S^2 (f=1/e)	4.95 [3.91, 5.81]	0.09 [-0.08, 0.27]	0.07 [0.00, 0.26]	-1.93 [-6.31, -0.48]
Classification (Entropy)	S_{all} (f=1/e)	12.59 [11.20, 13.85]	0.03 [-0.17, 0.20]	0.06 [0.00, 0.20]	15.05 [13.63, 15.94]
Classification (Entropy)	Rank-inspired	45.08 [39.63, 50.43]	5.81 [4.25, 7.59]	2.36 [1.85, 3.95]	70.23 [65.07, 73.62]
Classification (Entropy)	Prophet-style	17.23 [11.01, 23.36]	0.18 [-0.12, 0.48]	0.09 [0.00, 0.33]	42.00 [30.92, 50.87]
Classification (Entropy)	ERT ($m_{\text{try}} = 1$)	54.73 [48.68, 60.48]	8.26 [6.55, 10.11]	4.43 [3.19, 7.51]	95.82 [92.59, 97.24]
Classification (Entropy)	ERT ($m_{\text{try}} = p/3$)	49.27 [43.40, 54.90]	2.76 [2.04, 3.49]	1.80 [1.28, 2.60]	88.55 [82.45, 93.26]
Classification (Entropy)	ERT ($m_{\text{try}} = 2p/3$)	43.91 [38.27, 49.54]	1.07 [0.59, 1.54]	1.08 [0.67, 1.28]	83.77 [74.12, 89.77]
Classification (Entropy)	ERT ($m_{\text{try}} = p$)	38.61 [32.88, 44.36]	0.40 [0.02, 0.75]	0.49 [0.21, 0.64]	83.46 [77.84, 87.50]

Secretary-inspired split search is therefore not a universal substitute for exhaustive CART. On this benchmark, exact-feature stopping is the clearest conservative candidate for ensemble study, while ERT occupies a complementary speed-oriented regime. More broadly, the useful choice depends on task, loss tolerance, and whether reduced gain-evaluation effort converts into measured time on the target implementation.

Data and code availability

The source datasets are publicly available through PMLB. The retained-entry inventory, inferential summaries, implemented methods, and scripts used to run the benchmark and generate every reported figure and table are available in the GitHub repository. The archived per-seed benchmark outputs accompany the revision as supplementary data.

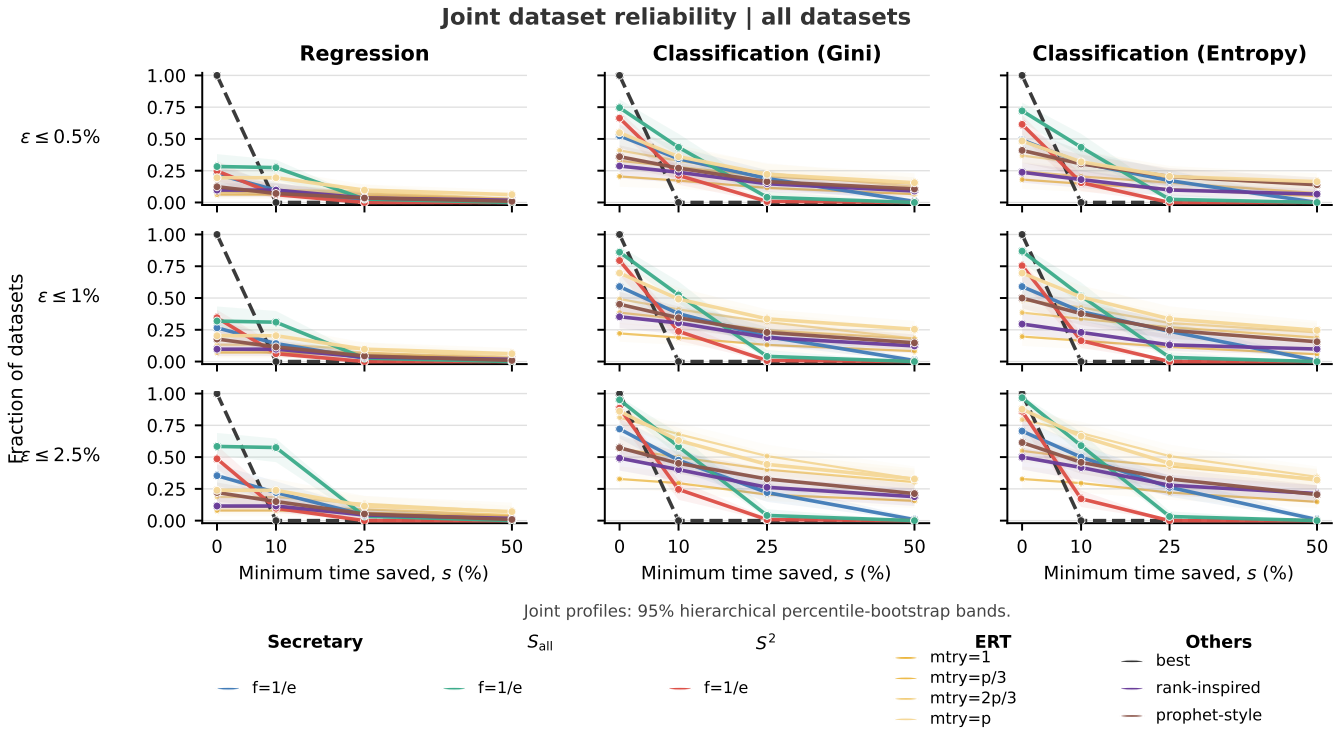

Fig. 2. Entry-level reliability under specified time-saving and predictive-loss constraints. Bands show 95% hierarchical bootstrap intervals for joint success.

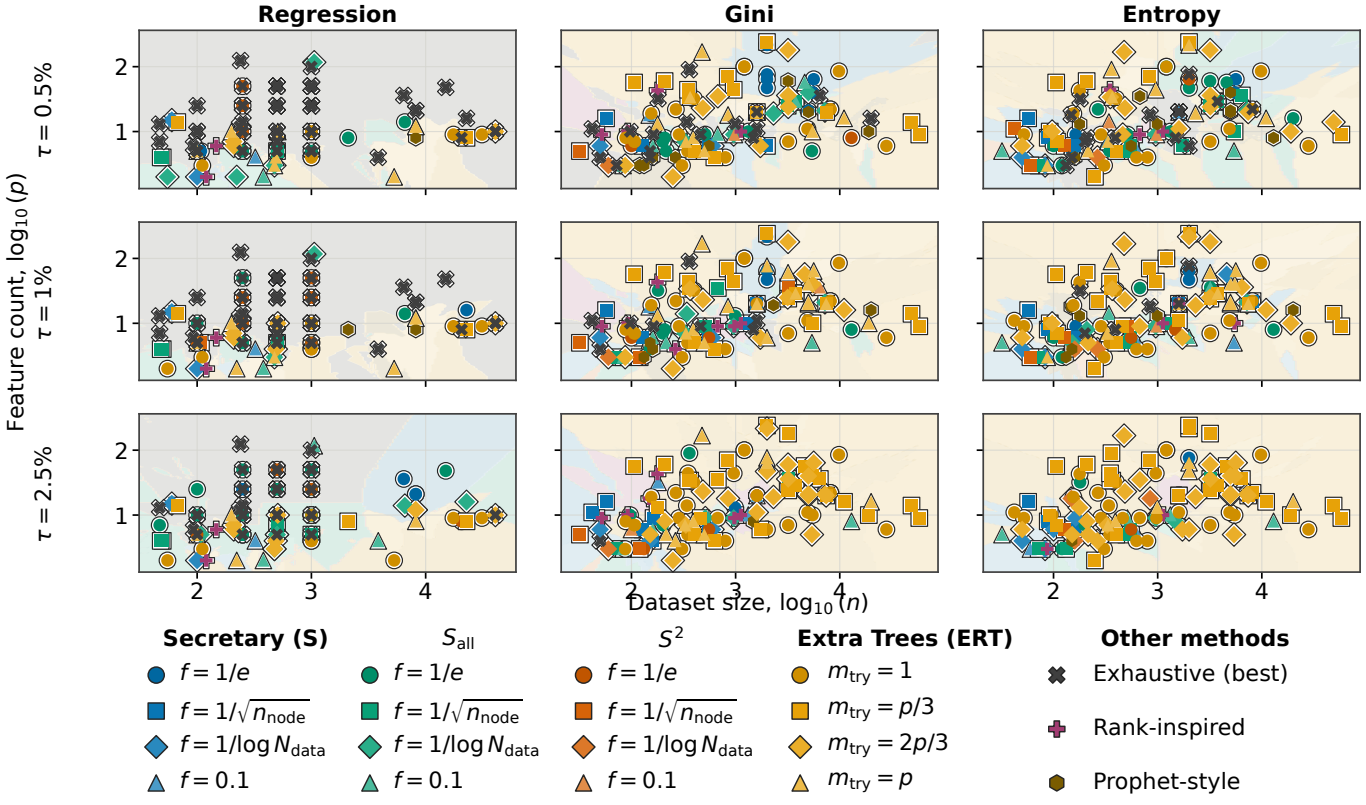

Fig. 3. Tolerance-constrained regime maps. Columns show regression, Gini classification, and entropy classification; rows show margins 0.5, 1, and 2.5, measured as bounded-relative-RMSE percentages for regression and weighted-F1 percentage points for classification. Each entry is colored by its fastest admissible method; pale regions are descriptive 7-nearest-neighbor interpolation.

Funding

This work was supported through the Research Chair DIALog under the aegis of the Risk Foundation, a joint initiative by Université Paris-Dauphine PSL and CNP Assurances. The author was solely responsible for the study design, analysis, interpretation, manuscript preparation, and decision to submit.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Mathias Valla: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review and editing, Visualization, and Project administration.

Acknowledgments

Work conducted within the Research Chair DIALog under the aegis of the Risk Foundation, a joint initiative by Université Paris-Dauphine PSL and CNP Assurances.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During preparation of this revision, the author used OpenAI Codex to assist with language editing, source-code review, and preparation of analysis scripts. The author reviewed and edited the resulting material as needed and takes full responsibility for the content of the article.

References

[1] L. Breiman, J. H. Friedman, R. A. Olshen, C. J. Stone, Classification and Regression Trees, Wadsworth, Belmont, CA, 1984.

[2] J. R. Quinlan, Induction of decision trees, Machine Learning 1 (1986) 81–106. doi:10.1007/BF00116251.

[3] U. M. Fayyad, K. B. Irani, On the handling of continuous-valued attributes in decision tree generation, Machine Learning 8 (1992) 87–102. doi:10.1007/BF00994007.

[4] L. Breiman, Random forests, Machine Learning 45 (1) (2001) 5–32. doi:10.1023/A:1010933404324.

[5] P. Geurts, D. Ernst, L. Wehenkel, Extremely randomized trees, Machine Learning 63 (1) (2006) 3–42. doi:10.1007/s10994-006-6226-1.

[6] R. Appel, T. Fuchs, P. Dollár, P. Perona, Quickly boosting decision trees—pruning underachieving features early, in: Proceedings of the 30th International Conference on Machine Learning, Vol. 28 of Proceedings of Machine Learning Research, 2013, pp. 594–602.

[7] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2016, pp. 785–794. doi:10.1145/2939672.2939785.

[8] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, LightGBM: A highly efficient gradient boosting decision tree, in: Advances in Neural Information Processing Systems, Vol. 30, 2017.

[9] P. Domingos, G. Hulten, Mining high-speed data streams, in: Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2000, pp. 71–80. doi:10.1145/347090.347107.

[10] M. Bressan, M. Sozio, Fully-dynamic approximate decision trees with worst-case update time guarantees, in: Proceedings of the 41st International Conference on Machine Learning, Vol. 235 of Proceedings of Machine Learning Research, 2024, pp. 4517–4541.

[11] H. Pham, H. Ta, H. T. Vu, Constructing decision trees from data streams, in: 2025 IEEE International Symposium on Information Theory, IEEE, 2025, pp. 1–6. doi:10.1109/ISIT63088.2025.11195218.

[12] N. Tatti, Approximating splits for decision trees quickly in sparse data streams, in: Proceedings of the 2025 SIAM International Conference on Data Mining, SIAM, 2025, pp. 647–655. doi:10.1137/1.9781611978520.69.

[13] J. P. Gilbert, F. Mosteller, Recognizing the maximum of a sequence, Journal of the American Statistical Association 61 (313) (1966) 35–73. doi:10.1080/01621459.1966.10502008.

[14] T. S. Ferguson, Who solved the secretary problem?, Statistical Science 4 (3) (1989) 282–296. doi:10.1214/ss/1177012493.

[15] P. Nuti, J. Vondrák, Secretary problems: The power of a single sample, in: Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SIAM, 2023, pp. 2015–2029. doi:10.1137/1.9781611977554.ch77.

[16] A. Rubinstein, J. Z. Wang, S. M. Weinberg, Optimal single-choice prophet inequalities from samples, in: 11th Innovations in Theoretical Computer Science Conference (ITCS 2020), Vol. 151 of LIPIcs, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 60:1–60:10. doi:10.4230/LIPIcs.ITCS.2020.60.

[17] D. Bertsimas, J. Dunn, Optimal classification trees, Machine Learning 106 (2017) 1039–1082. doi:10.1007/s10994-017-5633-9.

[18] V. Babbar, H. McTavish, C. Rudin, M. Seltzer, Near-optimal decision trees in a SPLIT second, in: Proceedings of the 42nd International Conference on Machine Learning, Vol. 267 of Proceedings of Machine Learning Research, 2025, pp. 2114–2175.

[19] J. D. Romano, T. T. Le, W. L. Cava, J. T. Gregg, D. J. Goldberg, P. Chakraborty, N. L. Ray, D. Himmelstein, W. Fu, J. H. Moore, Pmlb v1.0: an open-source dataset collection for benchmarking machine learning methods, Bioinformatics 38 (3) (2022) 878–880. doi:10.1093/bioinformatics/btab727.

[20] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. VanderPlas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, É. Duchesnay, Scikit-learn: Machine learning in python, Journal of Machine Learning Research 12 (2011) 2825–2830.

[21] J. Demšar, Statistical comparisons of classifiers over multiple data sets, Journal of Machine Learning Research 7 (2006) 1–30.

[22] B. Efron, Bootstrap methods: Another look at the jackknife, The Annals of Statistics 7 (1) (1979) 1–26. doi:10.1214/aos/1176344552.